TRABALHO 2 (INDIVIDUAL)

A Entrada

A entrada do algoritmo será basicamente um conjunto de retângulos e círculos dispostos numa região do plano cartesiano e, possivelmente, algumas consultas, por exemplo, que indagam se duas das formas geométricas se sobrepõem.

Considere a Figura 1. Cada forma geométrica é definida por uma coordenada âncora e por suas dimensões. A coordenada âncora do círculo é o seu centro e sua dimensão é definida por seu raio. A coordenada âncora do retângulo é seu canto inferior esquerdo[1] e suas dimensões são sua largura e sua altura. As coordenadas que posicionam as formas geométricas são valores reais e estão contidas dentro da região delimitada pelos cantos (0,0) e (x_{max},y_{max}). Cada forma geométrica é indentificada por um número inteiro.

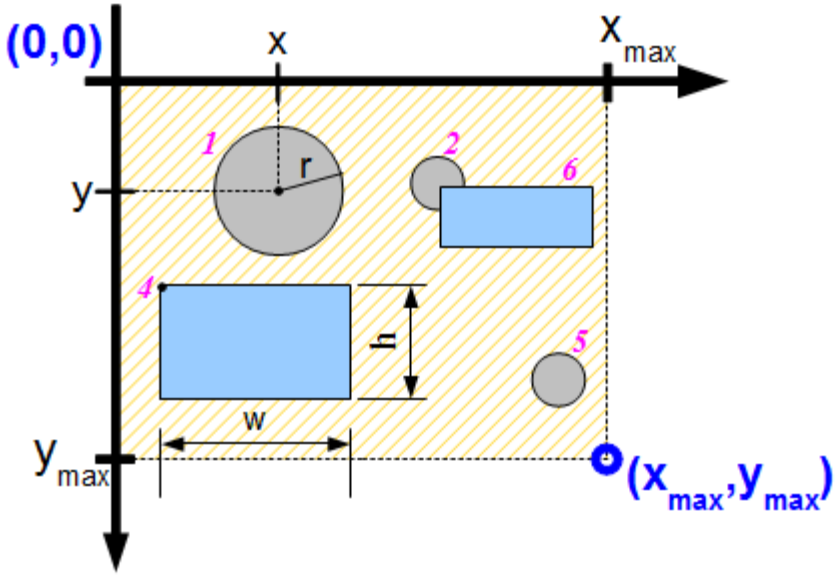


Figura 1: Elementos da entrada de dados.

A tabela abaixo mostra o formato do arquivo de entrada, composto, basicamente, por conjunto de comandos (uma operação por linha), a saber: **`c`** (desenhe um círculo), **`r`** (desenhe um retâgulo), **`o`** (consulta sobreposição), **`i`** (ponto é interno a figura?), **`d`** (calcule a distância entre duas figuras) e **`a`** (desenhe apenas as âncoras das figuras). A última linha possui a marca de final de arquivo (**`#`**).

Cada comando tem um certo número de parâmetros. Os parâmetros mais comuns são:

- `i,j,k`: número inteiro, maior ou igual a 1. Identificador de uma forma geométrica criada pelos comandos c ou r.
- `r`: número real. Raio do círculo.
- `x,y`: números reais. Coordenada (x,y).
- `cor`: string. Cor válida dentro do padrão SVG.[2]

1 Note que o plano cartesiano está desenhado "de ponta-cabeça" em relação à representação usual.
2 http://www.december.com/html/spec/colorsvg.html.
https://www.w3.org/Graphics/SVG/IG/resources/svgprimer.html

comando	parâmetros	descrição
c	`i r x y cor`	*desenhar círculo*
r	`i w h x y cor`	*desenhar retângulo: w é a largura do retângulo e h, a altura.*
o	`j k`	*As formas geométricas cujos indentificadores são j e k se sobrepõem?*[3]
i	`j x y`	*O ponto (x,y) é interno à j-ésima forma geométrica?*[4]
d	`j k`	*Qual é a distância entre os centro de massa das formas geométricas i e j?*
a	`sufixo cor`	*Cria um arquivo svg, mostrando apenas os centros dos círculos e os cantos dos retângulos criados pelos comandos c e r. O nome do arquivo gerado deve ser* nomebase-*sufixo.svg. Estes pontos devem ser desenhados usando a cor* `cor`

A figura abaixo mostra um exemplo de um arquivo de entrada (consistente com a Figura 1). Note que a extensão do arquivo é **`.geo`**. As primeiras operações desenham círculos e retângulos. Na parte parte final do arquivo, estão colocadas duas consultas de sobreposição. A primeira pergunta se o círculo 2 e o retângulo 6 se sobrepõem (de fato, sim) e, a segunda, pergunta se os círculos 1 e 5 se sobrepõem (de fato, não). O último comando solicita que seja produzido um outro arquivo **`.svg`** (a01-p1.svg) mostrando apenas os cantos dos retângulos e os centros dos círculos.

```
c 1 50.00 50.0 30.00 grey
r 6 121.0 46.0 100.0 30.0 cyan
c 2 120.0 45.0 15.0 grey
r 4 10.0 150.0 90.0 40.0  cyan
c 5 230.0 180.0 13.0 grey
o 2 6
o 1 5
a p1 red
#
```

***a01*.geo**

A Saída

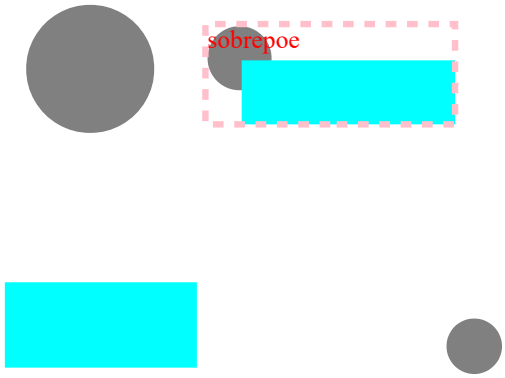
O programa deverá produzir um arquivo **`.svg`** e um arquivo **`.txt`** ambos com o mesmo nome base do arquivo **`.geo`**.

O arquivo .svg produzido deve mostrar as formas geométricas. Além disso, para o comando **`o`**, caso as figuras identificadas se sobreponham, elas devem ser envolvidas por um retâgulo tracejado contendo a palavra "sobrepoe". Existem várias ferramentas que renderizam arquivos .svg.
As figuras abaixo mostram um exemplo de arquivo **`.svg`** e sua respectiva renderização.

3 A borda da figura pertence à figura. Assim, as figuras que coincidem apenas nas bordas também se sobrepõem.
4 Um ponto na borda da figura pertence à figura, **mas não é interno** à figura.

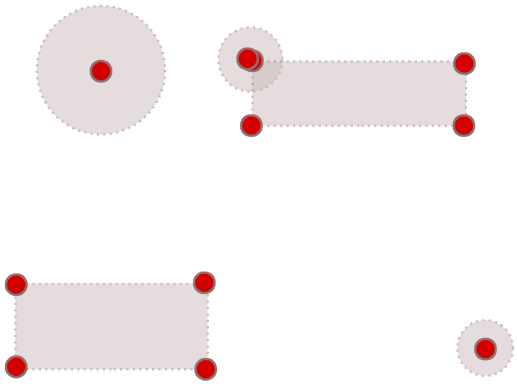
```
<svg height="200" width="250">
  <circle cx="50" cy="50" r="30"  fill="grey" />
  <circle cx="120" cy="45" r="15"  fill="grey"/>
  <rect    x="121"  y="46" width="100" height="30"  fill="cyan" />
  <rect    x="10"  y="150" width="90" height="40"  fill="cyan"/>
  <circle cx="230" cy="180" r="13" fill="grey" />
  <rect    x="104"  y="29" width="117" height="47" fill="none"
           style="stroke:pink;stroke-width:3;stroke-dasharray:5,5" />
  <text x="105" y="40" fill="red">sobrepoe</text>
</svg>
```

a01.svg



a01.svg (renderizado)

A operação **a** deve produzir um outro arquivo .svg, com nome *nomebase*`-sufixo.svg`, (no exemplo, `a01-p1.svg`) contendo apenas os cantos dos retângulos e centro dos círculos referentes aos comandos **c** e **r** processados até o momento, exemplificado na figura abaixo.[5]



a01-p1.svg

O processamento de um arquivo `.geo` deve também produzir um arquivo-texto contendo o resultado das consultas `i, d, o`. Este arquivo-texto deve ser nomeado `nome-base.txt`. Neste arquivo deve ser copiado em uma linha o texto da consulta e, na linha seguinte, o seu resultado.

5 A figura do exemplo apresenta as "sombras" dos círculos e retângulos apenas para facilitar o entendimento. No arquivo a01-p1.svg são apenas produzidos os pontos vermelhos.

O Programa

O nome do programa deve ser `t2` e aceitar dois parâmetros:

`t2 -f` *`arq`*`.geo -o` *`dir`* `-n` *`num`*

O primeiro parâmetro (**`-f`**) especifica o nome do arquivo de entrada e o segundo parâmetro (**`-o`**) indica o diretório onde os arquivos de saída (***`arq`*`.svg e` *`arq`*`.txt`**) deve ser colocado. Note que o nome do arquivo pode ser precedido por um caminho absoluto ou relativo e ***`dir`*** é um caminho absoluto ou relativo. O parâmetro **`-n`** informa o número máximo de operações **`r`** e **`c`** contidas no arquivo.

A seguir, alguns exemplos de possíveis invocações de **`t2`**:

- `t2 -f /home/ed/testes/t001.geo -o /home/ed/alunos/aluno1/o/ -n 100`
- `t2 -f /home/ed/testes/t001.geo -n 1000 -o /home/ed/alunos/aluno1/o`
- `t2 -n 10000 -f ./testes/t001.geo -o /home/ed/alunos/aluno1/o/`
- `t2 -f ./testes/t001.geo -n 100000 -o ./alunos/aluno1/o`

A Implementação

Os comandos de um arquivo devem ser mapeados em structs, alocadas dinamicamente. Ou seja, deverá existir um tipo de struct para armazenar os parâmetros dos comandos **`c`** e outro tipo para armazenar os parâmetros do comando **`r`**, etc. Sugere-se guardar os apontadores para tais estruturas num vetor: o identificador da forma geométrica pode corresponder ao índice do vetor.

O programa deve fazer bom uso de procedimentos e deve estar bem comentado.

É esperado que o programa seja dividido em módulos.

O Que Entregar

Submeter a sala Moodle o arquivo ***nome*.zip**. Note que ***nome*** é alguma abreviatura do nome do aluno. A abreviatura só deve conter letras minúsculas, sem acentuação (por exemplo, **`jlsilva`**, poderia se referir ao aluno José Luís da Silva. A organização do .zip deve estar de acordo com a descrição à frente.

A Avaliação

Espera-se uma atitude pró-ativa para a aquisição dos conhecimentos (i.e., estudo) para resolver o problema proposto.

A avaliação se baseará em três critérios: **(a)** compilação e testes do executável; **(b)** inspecção do código-fonte; **(c)** inspecção do arquivo tads.pdf.

A nota será proporcional ao número de testes executados com sucesso. Portanto, caso o programa não compile ou não execute corretamente, a nota poderá ser muito baixa!

PROCESSO DE COMPILAÇÃO E TESTES DO TRABALHO

Organização do ZIP a ser entregue

A organização do zip a ser entregue pelo aluno deve ser a seguinte:

[abreviatura-nome]

`LEIA-ME.txt`	*colocar matrícula e o nome do aluno. Atenção: O número da matricula de estar no início da primeira linha do arquivo. Só colocar os números; não colocar qualquer pontuação.*
`tads.pdf`	*Descrição de todos os tipos abstratos usados no projeto.*
`/src`	*(arquivos-fonte)*
`makefile`	*deve ter target para a geração do arquivo objeto de cada módulo e o target* **`t2`** *que produzirá o executável de mesmo nome dentro do mesmo diretório* **`src`**. *Os fontes devem ser compilados com as opções* `-pedantic -ansi`.
`*.h e *.c`	***Atenção:*** *não devem existir outros arquivos além dos arquivos fontes e do makefile*

Organização do diretório para a compilação e correção dos trabalhos (no computador do professor):

[HOME_DIR]

`*.py`	*scripts para compilar e executar*
`\t`	*diretório contendo os arquivos de testes*
`*.geo`	*arquivos de teste*
`\alunos`	*(contém um diretório para cada aluno)*
`\abrnome`	*diretório pela expansão do arquivo submetido*
	outros subdiretórios para os arquivos de saída informados na opção `-o`

Os passos para correção serão os seguintes:

1. O arquivo .zip será descomprimido dentro do diretório alunos, conforme mostrado acima
2. O makefile provido pelo aluno será usado para compilar os módulos e produzir o executável. Os executáveis devem ser produzidos no mesmo diretório dos arquivos fontes O professor usará o GNU Make. Serão executadas (a partir dos scripts) o seguinte comando:
   - **`make t2`**
3. O programa será executado automaticamente várias vezes: uma vez para cada arquivo de testes e o resultado produzido será inspecionado visualmente pelo professor. Cada execução produzirá (pelo menos) um arquivo **`.svg`** diferente dentro do diretório informado na opção **`-o`**. Possivelmente serão produzidos outros arquivos `.svg` (comando `a`) e um arquivo .txt (comandos `i, d, o`)